Trabalho Prático de Avaliação (TPA1) – CodeRunner

Objetivo: Usar gRPC para construir um sistema com vários servidores que repartem a carga de múltiplos clientes. Cada pedido é executado num *container* Docker isolado, lançado dinamicamente pelo servidor.

Notas prévias

- Peso: 20% da avaliação final.
- Entrega no Moodle: um Zip com o código fonte e o projeto gradle, sem as diretorias `build/` e `.gradle/`, e um *readme* com os pressupostos e as instruções para executar o sistema.
- Apresentação e demonstração: 10 a 15 minutos por grupo, na semana de 9 de novembro de 2026.
- **Cada grupo tem de fazer a entrega no Moodle até 8 de novembro de 2026 (23:59h).**

Cenário

O sistema ***CodeRunner*** é uma plataforma de ensino de programação: os utilizadores submetem programas Python e obtêm o resultado da execução. O código submetido não é de confiança, por isso cada execução corre num *container* efémero, sem rede e com limites de memória e de tempo. Existem vários servidores *RunServer* para distribuir a carga dos clientes (Figura 1).

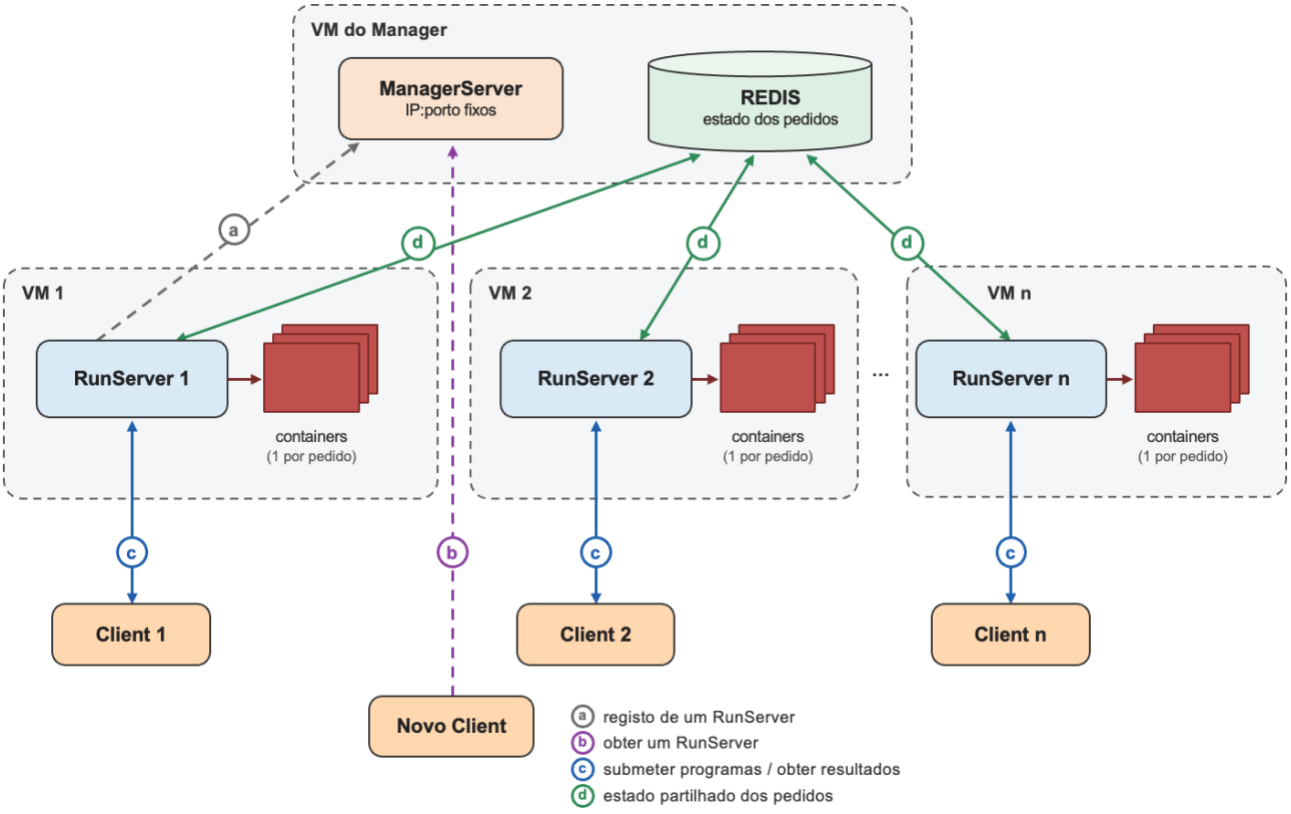


Figura 1 – Arquitetura do sistema *CodeRunner*

Requisitos funcionais

Componentes

- **ManagerServer** (IP e porto fixos e conhecidos): Oferece dois contratos:
  - **a – Registo:** um *RunServer* indica o seu IP e porto e recebe o endereço do REDIS.
  - **b – Obter servidor:** um *Client* recebe o IP e porto de um *RunServer*. Os clientes são distribuídos com equilíbrio pelos servidores registados nesse momento.

- **RunServer**: Oferece o contrato **c – Execução**, com duas operações:
  - *submeter*: recebe por *streaming* de cliente os bytes de um programa (*.py*) e, opcionalmente, um ficheiro de *input* (*stdin*); retorna de imediato o identificador do pedido. O servidor executa o programa e guarda o resultado num ficheiro local, out.txt, e qualquer erro no ficheiro err.txt.
  - *obter resultado*: dado um identificador, devolve o estado do pedido e, se já existir, o resultado e erro. O resultado do programa é retornado por *stream* de servidor. Note que cada RunServer só tem os *outputs* dos programas que executou.
- **REDIS** (num *container* na VM do *ManagerServer*): **d -** Guarda o estado de todos os pedidos. O *stdout* e o *stderr* **não** são guardados no REDIS: ficam no *RunServer* que executou o pedido.
- **Client.** O *Client* deve conseguir obter o resultado de um pedido a partir do seu identificador, mesmo que esteja ligado a outro *RunServer* ou seja uma nova instância.

Ciclo de vida de um pedido para executar programa Python

O *RunServer* recebe um programa, gera um identificador único (UUID), regista o pedido no REDIS com o estado PENDENTE e devolve o identificador ao *Client*, sem esperar pela execução. A Figura 2 mostra todo o ciclo de um pedido deste tipo.

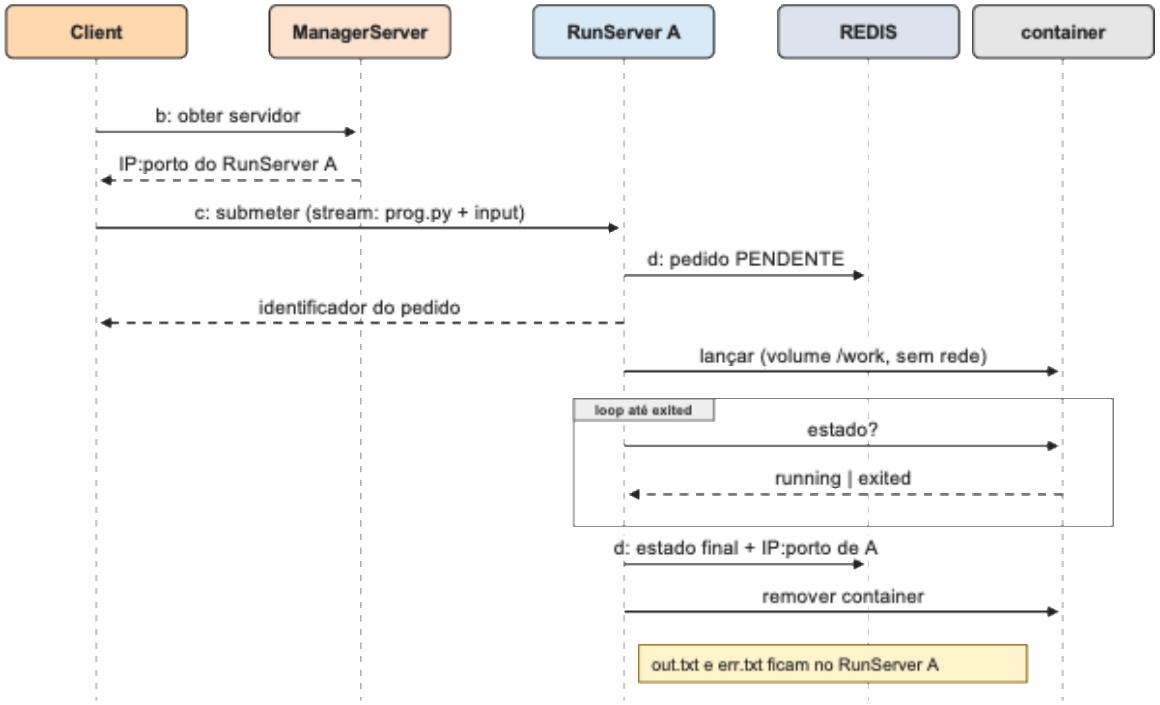


Figura 2 – Ciclo de vida de um pedido para execução de programa Python. Os ficheiros de resultado ficam no RunServer que executou o pedido. A figura não mostra o pedido de consulta de resultado.

Requisitos não funcionais

- Contratos separados (*loose coupling*): *ManagerServer–RunServer* (**a**), *ManagerServer–Client* (**b**) e *RunServer–Client* (**c**).
- O *ManagerServer* nunca falha. Não é preciso tratar falhas de *RunServer*, mas podem ser adicionados servidores a qualquer momento, incluindo durante a demonstração.
- Nenhum *container* de pedido pode ficar por remover, mesmo em caso de erro.
- Os resultados da execução guardados em cada servidor têm de ser removidos quando já não forem necessários. Sugere-se que, após a obtenção do resultado por parte de um cliente, o resultado (output.txt, error.txt e outras estruturas) seja apagado do servidor.
- Cada *RunServer* atende pedidos concorrentes: o acesso ao REDIS e a gestão dos *containers* têm de ser seguros face à concorrência.
- Cada VM aloja um *RunServer* e deve ter a imagem obtida previamente (*docker pull python:3.12-slim*).

- Demonstração: pelo menos 3 *RunServer* em pelo menos 2 VM, com pelo menos um *Client* ligado a cada servidor. Os *Client* podem correr nas máquinas dos alunos ou nas VM.
- Aplicação *Demo.jar <argumentos>*: simula vários clientes concorrentes, com programas que terminam normalmente, que terminam com erro e que não terminam.

Valorização (opcional)

- Limitar a N os *containers* em simultâneo em cada *RunServer*; os restantes pedidos ficam em fila, com estado PENDENTE.
- Limitar o CPU de cada *container*.

Sugestões gerais

- Desenhe primeiro a arquitetura, os contratos *protobuf* e os principais diagramas de interação.
- Trate e propague as exceções, para que o sistema tolere algumas falhas.
- Esclareça dúvidas com os professores e consulte as FAQ no Moodle.

Anexo 1 – Projeto REDIS

Em https://github.com/isel-distributed-computing/CD-public/tree/main/Trabs/TP1/1-redis-kv-example está disponível um exemplo em Java para escrita e leitura de valores na base de dados REDIS. Estão também disponíveis scripts para lançar o contentor com a base de dados.

Anexo 2 – Execução de programa Python em contentor

Em https://github.com/isel-distributed-computing/CD-public/tree/main/Trabs/TP1/2-docker-python estão disponíveis scripts que exemplificam a execução de contentores para correr os programas Python num ambiente isolado. Analise os comandos Docker e experimente os scripts.

```
docker run --name run-<id> --network none --memory 256m --memory-swap 256m --user <uid>:<gid> \
-v <diretoria do pedido>:/work python:3.12-slim \
sh -c "timeout 10 python /work/prog.py < /work/input.txt > /work/out.txt 2> /work/err.txt; echo
\$? > /work/exit.txt"
```

Este comando Docker realiza as seguintes ações:

- Cria e arranca um contentor chamado `run-<id>`, a partir da imagem `python:3.12-slim`.
- Isola o contentor da rede (`--network none`): o programa não consegue fazer ligações para o exterior.
- Limita a memória a 256 MB, sem swap adicional (`--memory` e `--memory-swap`). Se o programa exceder o limite, é terminado (código 137).
- Executa o programa sem privilégios de root (`--user`), com o mesmo utilizador e grupo que lança o contentor.
- Configura a diretoria do pedido em `/work` (`-v`). É através deste mecanismo que o programa e os dados ficam disponíveis ao contentor e os resultados são gravados no *host*.
- Executa prog.py com um limite de 10 segundos (`timeout 10`). Se o tempo for excedido, o programa é terminado com o código 124.
- Redireciona as entradas e saídas para ficheiros: o `input.txt` é usado como *stdin*, o *stdout* vai para `out.txt` e o *stderr* para `err.txt`.
- Guarda o código de saída em `exit.txt`

O estado do pedido é obtido a partir de `exit.txt`: 0 é CONCLUIDO, 124 é TIMEOUT e qualquer outro valor é ERRO.

Estes comandos serão realizados programaticamente pelo `RunServer`, usando a biblioteca `docker-java`.

Anexo 3 – Biblioteca Java para lançamento de contentores e monitorização de estado

Em https://github.com/isel-distributed-computing/CD-public/tree/main/Trabs/TP1/3-docker-java-examples existem dois exemplos de utilização da biblioteca docker-java (https://github.com/docker-java/docker-java).

O primeiro exemplo, `docker-java-busybox`, é um projeto Gradle onde é executado um contentor a partir da imagem `busybox`, aguarda a conclusão do contentor e apresenta o resultado no *standard output*.

O segundo exemplo, `docker-java-runner`, é um projeto Gradle onde é executado um contentor de forma equivalente à linha de comandos do Anexo 2. Neste exemplo, os ficheiros de entrada e saída estão na diretoria `request` na base do projeto.

Altere o programa usado pelo exemplo docker-java-runner para executar cada um dos outros programas, `loop.py` e `memory.py`, os quais terminam com erro. O `loop.py` termina com erro de *timeout* por ser um ciclo infinito, o `memory.py` termina porque tenta alocar memória em excesso.